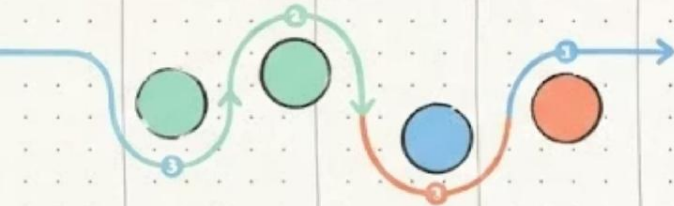


Gestão do Ciclo de Vida da Aplicação

Design Patterns (Parte 3)

Professor Romário da Silva Borges



Comportamentais

The background features a light blue gradient with several wavy, translucent lines in shades of purple and blue. These lines are overlaid with faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and Java. The code is mostly illegible due to the low opacity and the wavy motion blur effect.

Os padrões comportamentais são padrões que propõem soluções flexíveis para interação e divisão de responsabilidades entre classes e objetos. São eles: Strategy, Observer, Template Method, Visitor, Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento e State.

7 - Strategy

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Faint, semi-transparent snippets of code are visible, particularly on the left and right sides, suggesting a technological or digital theme. The overall aesthetic is modern and clean.

Contexto: Biblioteca de Estruturas de Dados

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    public void sort() {  
        ... // ordena a lista usando Quicksort  
    }  
  
}
```

Problema

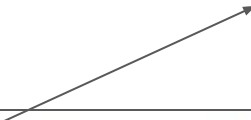
- Classe MyList **não** está aberta a extensões
- Possível extensão: mudar o algoritmo de ordenação
- Usar ShellSort, HeapSort, etc

Solução: Padrão Strategy

- Objetivo:
 - Parametrizar os algoritmos usados por uma classe
 - Tornar uma classe "aberta" a novos algoritmos
- No exemplo: novos algoritmos de ordenação

Passo #1: Criar uma hierarquia de "estratégias"
(estratégia = algoritmo)

SortStrategy poderia ser também uma interface



```
abstract class SortStrategy {  
    abstract void sort(MyList list);  
}  
  
class QuickSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}  
  
class ShellSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}
```

Passo #2: Modificar MyList para usar a hierarquia de estratégias

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
  
}
```

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
  
}
```

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
  
}
```

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
}
```

8 - Observador

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement. Faint, illegible snippets of code are visible, particularly on the left and right sides, suggesting a technical or programming theme.

Contexto: Sistema de uma Estação Meteorológica

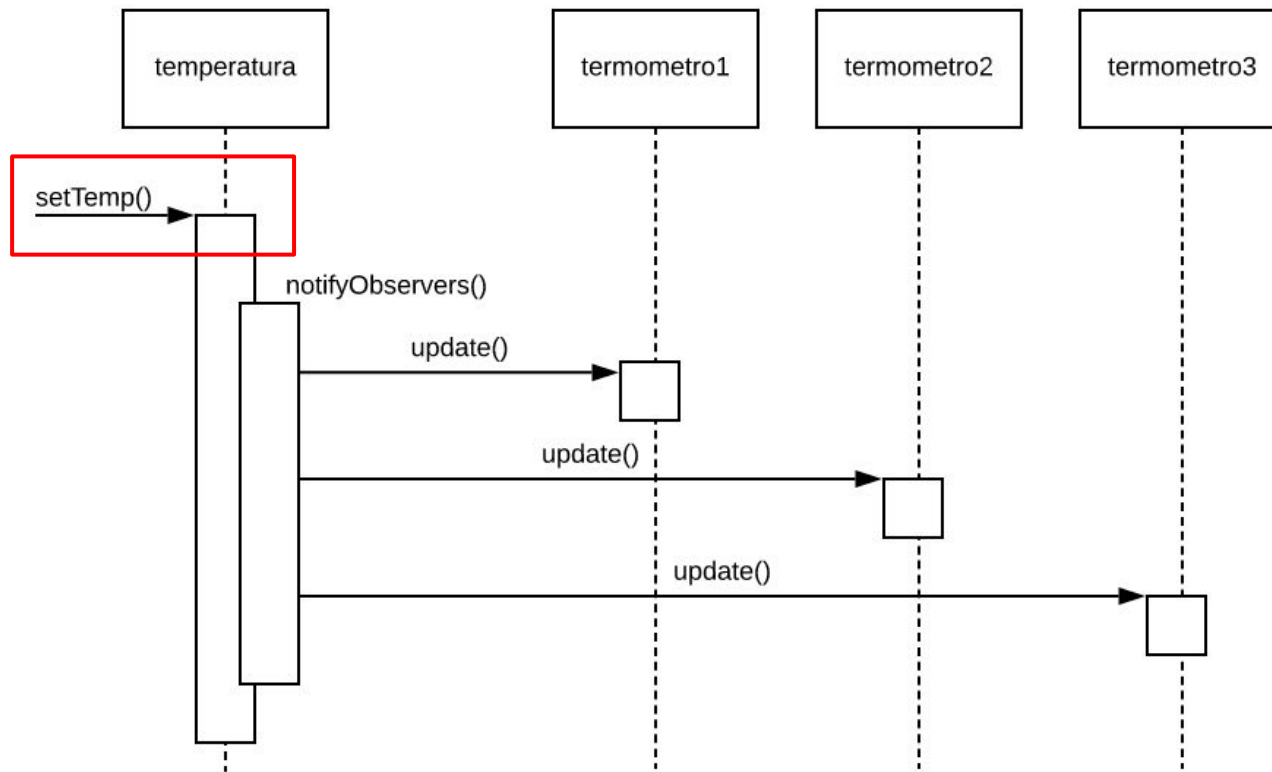
- Duas classes principais:
 - Temperatura
 - Termômetro
- Diversos termômetros: digital, analógico, web, celular, etc
- Se a temperatura mudar, os termômetros devem ser atualizados

Problema

- Não queremos acoplar Temperatura a Termômetros
- Motivos:
 - Tornar classe de dados (modelo) independente de classes de visão (interface com usuários)
 - Tornar flexível a adição de um novo tipo de termômetro no sistema

Solução: **Padrão Observador**

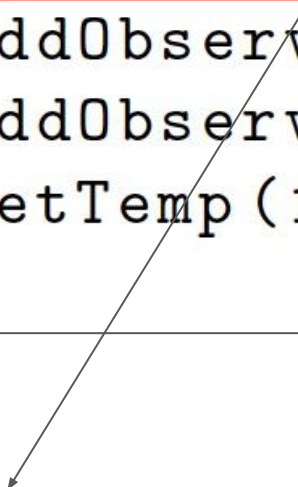
- Implementa uma relação do tipo um-para-muitos entre os seguintes objetos
 - Sujeito (Temperatura)
 - Observadores (Termômetros)
- Quando o estado de um Sujeito muda, seus Observadores são notificados.
- Mas Sujeito não conhece o tipo concreto de seus Observadores



Programa Principal

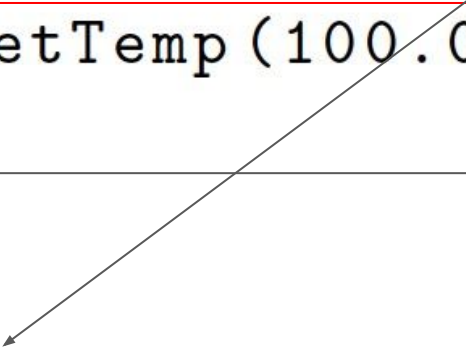
```
void main() {  
    Temperatura t = new Temperatura();  
    t.addObserver(new TermometroCelsius());  
    t.addObserver(new TermometroFahrenheit());  
    t.setTemp(100.0);  
}
```

Sujeito



Programa Principal

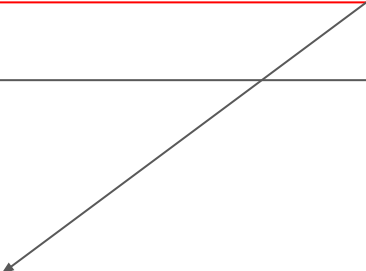
```
void main() {  
    Temperatura t = new Temperatura();  
    t.addObserver(new TermometroCelsius());  
    t.addObserver(new TermometroFahrenheit());  
    t.setTemp(100.0);  
}
```



Dois observadores

Programa Principal

```
void main() {  
    Temperatura t = new Temperatura();  
    t.addObserver(new TermometroCelsius());  
    t.addObserver(new TermometroFahrenheit());  
    t.setTemp(100.0);  
}
```



Notifica os termômetros

Classe Temperatura

```
class Temperatura extends Subject {  
  
    private double temp;  
  
    public double getTemp() {  
        return temp;  
    }  
  
    public void setTemp(double temp) {  
        this.temp = temp;  
        notifyObservers();  
    }  
  
}
```

Classe que
implementa
addObservers e
notifyObservers

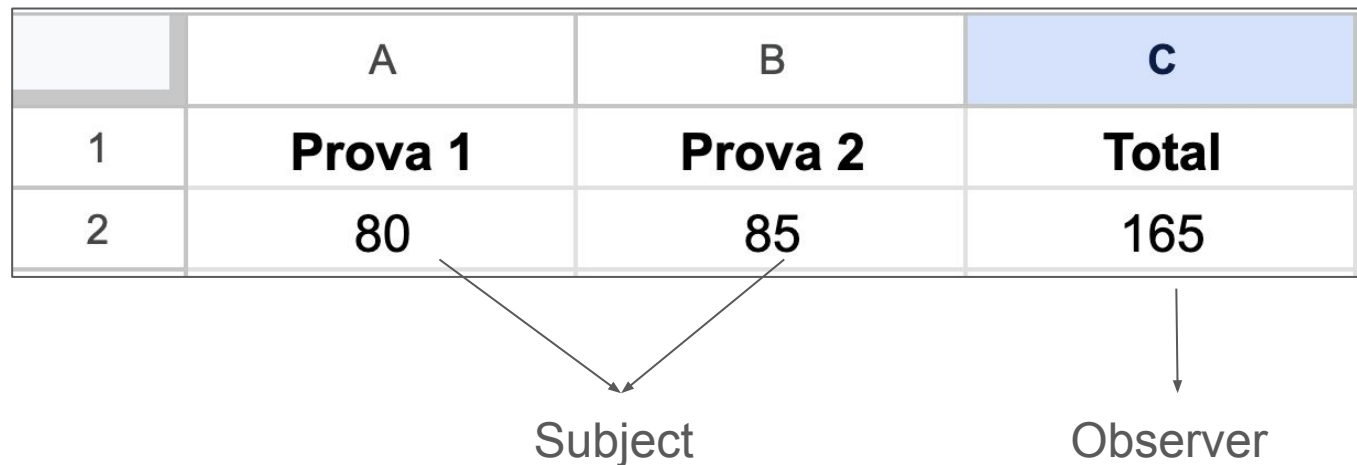
Uma classe Termômetro

```
class TermometroCelsius implements Observer{  
  
    public void update(Subject s){  
        double temp = ((Temperatura) s).getTemp();  
        System.out.println("Temperatura Celsius: " + temp);  
    }  
  
}
```

Todos Observadores devem implementar esse método.
Chamada de **notifyObservers** (em Temperatura) resulta na execução de **update** de cada um de seus observadores

Outro Exemplo: Planilhas

- Quando o valor de uma célula X muda, todas as células Y que fazem uso de X são automaticamente atualizadas



```
class Cell:
    def __init__(self):
        self.value = None
        self.observers = []

    def set_value(self, new_value):
        self.value = new_value
        self.notify_observers()

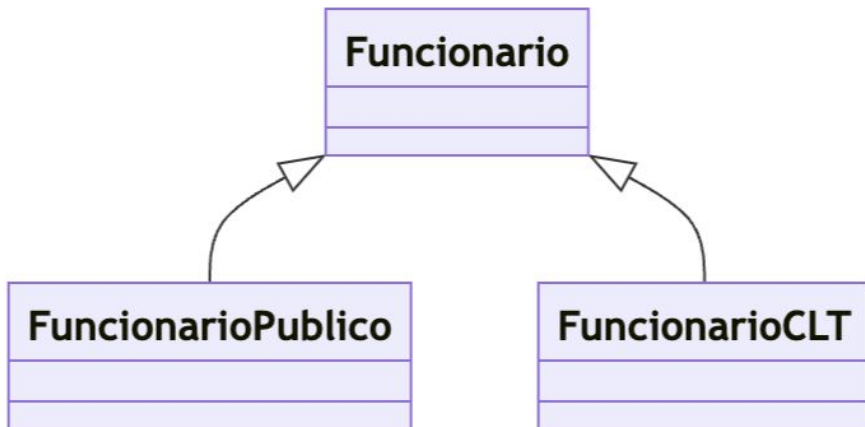
    def notify_observers(self):
        for observer in self.observers:
            observer.update(self)

    def add_observer(self, observer):
        self.observers.append(observer)
```

9 - Template Method

The background of the slide features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and C++, which are slightly blurred to maintain focus on the title.

Contexto: Folha de Pagamento



Problema

- Função que calcula salário de funcionários:
 - Passos são semelhantes para func. públicos e CLT
 - Porém, existem alguns detalhes diferentes
- Na classe pai (Funcionario) queremos definir o "workflow principal" (ou o template) para cálculo de salários
- E deixar aberto para as subclasses os refinamentos desses passos

Solução: Padrão **Template Method**

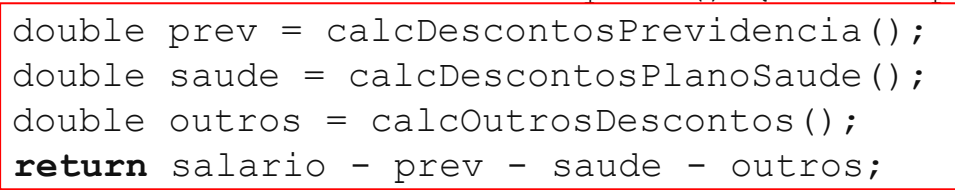
- Especifica como implementar o esqueleto de um algoritmo em uma classe abstrata X
- Mas deixando pendente alguns passos (ou métodos abstratos) para serem implementados nas subclasses
- Permite que subclasses customizem um algoritmo, mas sem mudar sua estrutura

```
abstract class Funcionario {  
  
    double salario;  
  
    ...  
    abstract double calcDescontosPrevidencia();  
    abstract double calcDescontosPlanoSaude();  
    abstract double calcOutrosDescontos();  
  
    public double calcSalarioLiquido() { // template method  
        double prev = calcDescontosPrevidencia();  
        double saude = calcDescontosPlanoSaude();  
        double outros = calcOutrosDescontos();  
        return salario - prev - saude - outros;  
    }  
}
```

Implementados
pelas
subclasses



```
abstract class Funcionario {  
  
    double salario;  
  
    ...  
    abstract double calcDescontosPrevidencia();  
    abstract double calcDescontosPlanoSaude();  
    abstract double calcOutrosDescontos();  
  
    public double calcSalarioLiquido() { // template method  
        double prev = calcDescontosPrevidencia();  
        double saude = calcDescontosPlanoSaude();  
        double outros = calcOutrosDescontos();  
        return salario - prev - saude - outros;  
    }  
}
```

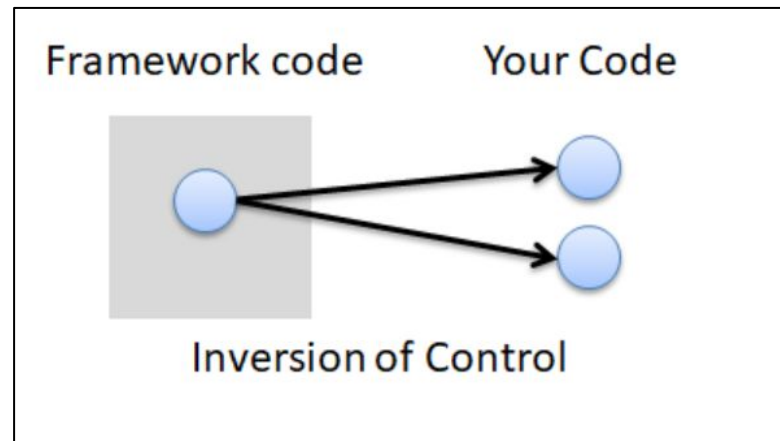
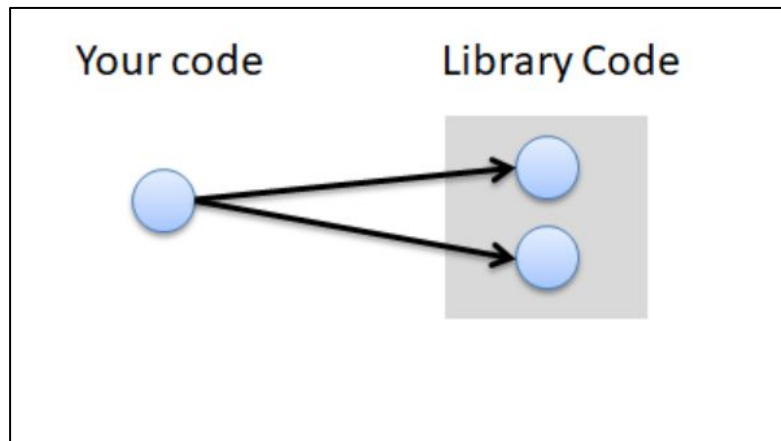


Passos
principais (ou
template, ou
modelo) para
cálculo de
salário líquido

Inversão de Controle

- Template Method é usado para implementar **Inversão de Controle**, principalmente em frameworks
- Framework: define o "modelo" de um algoritmo/sistema
 - Mas clientes podem parametrizar alguns passos
 - Framework chama esse código dos clientes
 - Daí o termo inversão de controle

Frameworks vs Bibliotecas

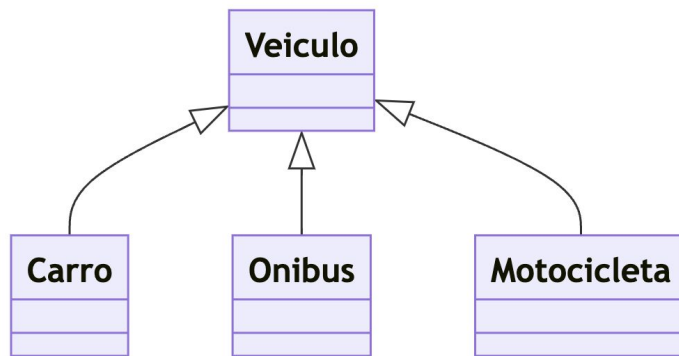


Fonte: <https://github.com/prmr/SoftwareDesign/blob/master/modules/Module-06.md>

10 – Visitor

The background features a series of fluid, wavy lines in shades of blue and purple that sweep across the frame. Interspersed within these waves are faint, semi-transparent snippets of code, likely from a programming language like JavaScript or Java, which are partially obscured by the motion of the lines.

Contexto: Veiculo e subclasses



E uma lista polimórfica de Veiculos

```
List<Veiculo> list = new ArrayList<Veiculo>();  
list.add(new Carro(..));  
list.add(new Onibus(..));  
...
```

Por que essa lista é chamada de polimórfica?

Problema

- Temos que realizar certas operações com os veículos da lista
- Exemplo:
 - Imprimir dados de veículos (sejam `Carro`, `Onibus`,...)
 - Salvar dados de veículos em disco
 - Enviar uma msg para os donos dos veículos
 - etc

Problema

- Queremos seguir o princípio Aberto/Fechado:
 - Manter `Veiculo` e suas subclasses fechados para mudanças, mas aberto para extensões
 - Extensões: operações que queremos realizar com veículos

Possível alternativa

```
interface Visitor {  
    void visit(Carro c);  
    void visit(Onibus o);  
    void visit(Motocicleta m);  
}  
  
class PrintVisitor implements Visitor {  
    public void visit(Carro c) { "imprime dados de carro" }  
    public void visit(Onibus o) { "imprime dados de onibus" }  
    public void visit(Motocicleta m) {"imprime dados de moto"}  
}
```

Interface foi criada porque amanhã poderemos ter outros tipos de Visitors
(exemplo: que salva os dados em um arquivo JSON)

Possível alternativa (cont.)

```
PrintVisitor visitor = new PrintVisitor();  
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {  
    visitor.visit(veiculo); // erro de compilação  
}
```

Possível alternativa (cont.)

```
PrintVisitor visitor = new PrintVisitor();  
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {  
    visitor.visit(veiculo); // erro de compilação  
}
```

- Em Java e linguagens similares, compilador não conhece o tipo dinâmico do parâmetro `veiculo`
- Logo, ele não sabe identificar qual dos três métodos `visit` de `PrintVisitor` deverá ser chamado

Decisão do método a ser chamado é "bi-dimensional"

	PrintVisitor	MailVisitor	SaveVisitor	etc
Carro				
Onibus				
Motocicleta				
etc				

Solução: **Padrão Visitor**

- Permite adicionar uma operação genérica em uma família de classes, sem mexer no código delas
- Simula multiple dispatching, em linguagens que oferecem apenas single dispatching

```
abstract class Veiculo {
    abstract public void accept(Visitor v);
}

class Carro extends Veiculo {
    ...
    public void accept(Visitor v) {
        v.visit(this);
    }
    ...
}

class Onibus extends Veiculo {
    ...
    public void accept(Visitor v) {
        v.visit(this);
    }
    ...
}

// Idem para Motocicleta
```

```
PrintVisitor visitor = new PrintVisitor();  
foreach (Veiculo veiculo: listaDeVeiculosEstacionados) {  
    veiculo.accept(visitor);  
}
```

Chama accept do tipo dinâmico de `veiculo`. Suponha que seja `Carro`

```
class Carro extends Veiculo {  
    ...  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
    ...  
}
```

Tipo de `this` é conhecido estaticamente (`Carro`)

A escolha do método ocorre em dois passos dinâmicos, mais um passo estático

veiculo.accept(visitor) ⇒

se "veiculo"
Carro
Onibus
Motocicleta
etc

Decisão apenas em tempo de execução

⇒ Carro.accept(visitor) { visitor.visit(this); }

⇒ Onibus.accept(visitor) { visitor.visit(this); }

⇒ Motocicleta.accept(visitor) { visitor.visit(this); }

Decisão em tempo de compilação. Tipo de "this" é conhecido estaticamente

visitor.visit(this) ⇒

se "visitor"	se "this"		
	Carro	Onibus	Motocicleta
PrintVisitor	visit(Carro)	visit(Onibus)	visit(Motocicleta)
etc			

Visitor: vantagens & desvantagens

Visitor: Vantagens

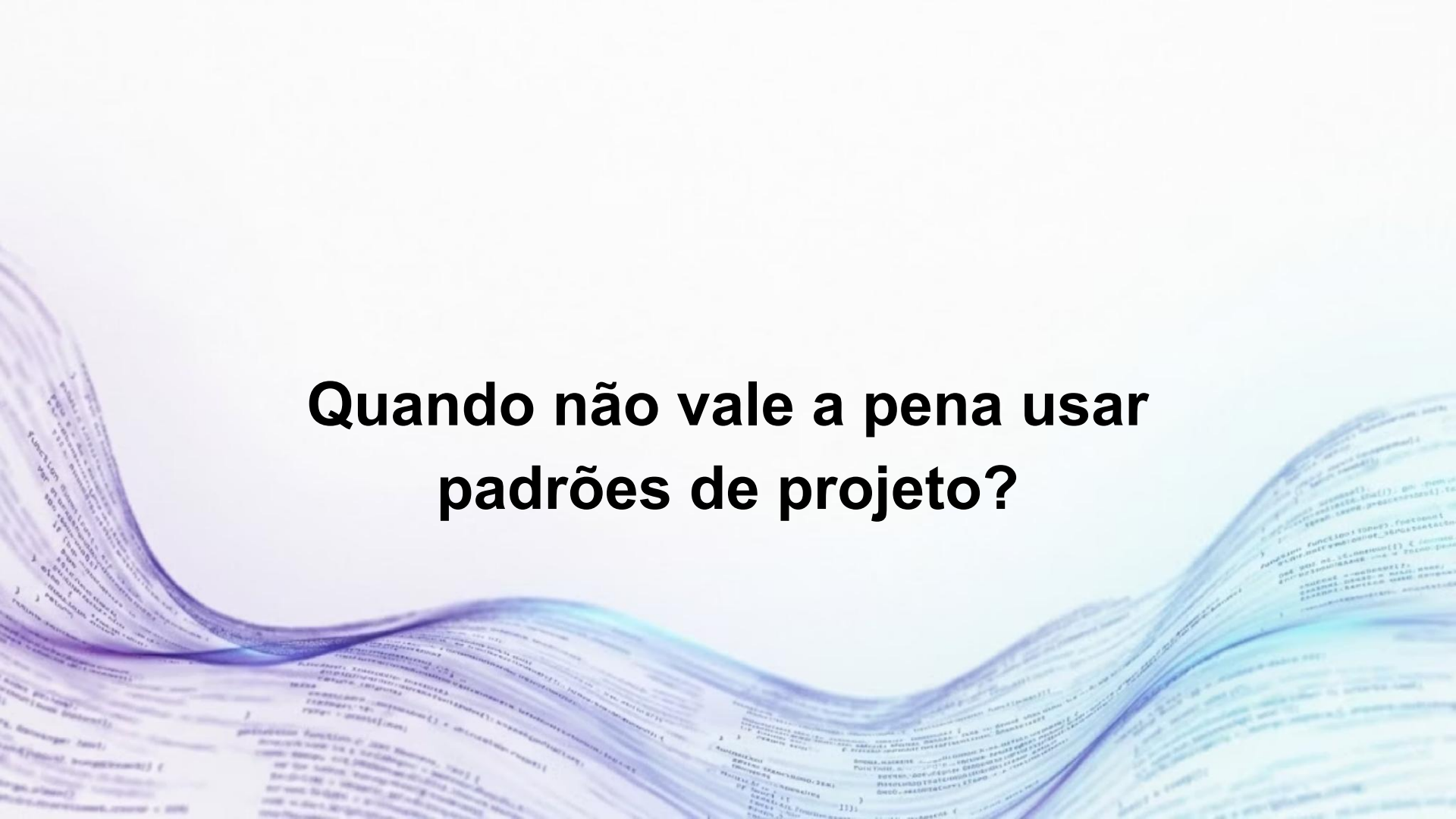
- Visitors facilitam a adição de um método em uma hierarquia de classes
- Pode existir um segundo Visitor, com outras operações
 - Exemplo: calcular imposto de veículos de MG

Visitor: Desvantagens

- A adição de uma nova classe na hierarquia (exemplo: `Caminhao`), implica que todos os `Visitors` terão que ser atualizados, com um novo método: `visit(Caminhao)`

Visitor: Desvantagens

- Visitors podem quebrar encapsulamento
 - Por exemplo, `Veiculo` pode ter que implementar métodos públicos expondo seu estado interno para que os Visitors tenham acesso a ele



**Quando não vale a pena usar
padrões de projeto?**

Padrões de Projeto $\Leftrightarrow$ Design for change

- Se a chance de mudanças é zero, não precisamos de padrões de projeto

Trade-off: Design for Change vs Complexidade

- Para viabilizar "design for change", Design Patterns complicam um pouco o projeto
- Por exemplo, demandam a criação de classes extras

Exemplo: Strategy

Solução sem padrão de projeto

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    public void sort() {  
        ... // ordena a lista usando Quicksort  
    }  
  
}
```

1 classe

Solução sem padrão de projeto

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    public void sort() {  
        ... // ordena a lista usando Quicksort  
    }  
  
}
```

1 classe

Solução usando Strategy

```
class MyList {  
  
    ... // dados de uma lista  
    ... // métodos de uma lista: add, delete, search  
  
    private SortStrategy strategy;  
  
    public MyList() {  
        strategy = new QuickSortStrategy();  
    }  
  
    public void setSortStrategy(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void sort() {  
        strategy.sort(this);  
    }  
  
}
```

1 classe (com mais código)
+1 classe abstrata
+2 classes de estratégias

```
abstract class SortStrategy {  
    abstract void sort(MyList list);  
}  
  
class QuickSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}  
  
class ShellSortStrategy extends SortStrategy {  
    void sort(MyList list) { ... }  
}
```

Paternite: um (provável) exemplo

org.springframework.aop.framework

Class `AbstractSingletonProxyFactoryBean`

[java.lang.Object](#)

└ [org.springframework.aop.framework.ProxyConfig](#)

└ [org.springframework.aop.framework.AbstractSingletonProxyFactoryBean](#)

All Implemented Interfaces:

[Serializable](#), [BeanClassLoaderAware](#), [FactoryBean](#), [InitializingBean](#)

Direct Known Subclasses:

[TransactionProxyFactoryBean](#)

```
public abstract class AbstractSingletonProxyFactoryBean
extends ProxyConfig
implements FactoryBean, BeanClassLoaderAware, InitializingBean
```

Convenient proxy factory bean superclass for proxy factory beans that create only singletons.

Manages pre- and post-interceptors (references, rather than interceptor names, as in [ProxyFactoryBean](#)) and provides consistent interface management.

Exercício

1. Cite três design patterns que ajudam nitidamente a tornar uma classe "aberta" para extensões? Isto é, favorecem um design que segue o Princípio Aberto/Fechado.

Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

